

17/02/2026 (8:25)

Was late

ORDER BY statement

Products

→ Arrange in descending order (price)

Product_id	Product_name	Price
111	Apple	2
112	Banana	4
113	Orange	3
114	Kiwi	5

SELECT product_name,
price

FROM products

Ord.

ORDER BY price DESC;

Product_name	Price
Kiwi	5
Banana	4
Orange	3
Apple	2

SELECT product_name,
price

FROM products

ORDER BY price ASC;

Product_name	Price
Apple	2
Orange	3
Banana	4
Kiwi	5

Limit statement → We ^{use} it to limit the number of rows

- Specify the number of rows that you want to see
- To view the structure of the table
- To take a snapshot to the table

SELECT *
FROM products
LIMIT 2;

Product_id	Product_name	Price
111	Apple	2
112	Banana	4

SELECT DISTINCT

↓ means different values in that column
means unique values

SELECT DISTINCT column_name
FROM database.schema.table;

SELECT DISTINCT subject
FROM curro.students.marks

→ Removes duplicates

↓

Subject
Maths
Geo

SQL Aliases (AS) → Renaming the column name.
→ Renaming the table name.

Products

Product_id	product_name	Price
111	Apple	2
112	Banana	4
113	Orange	3
114	Kiwi	5

SELECT product_id AS barcode,
product_name AS fruit_name,
price

FROM products;

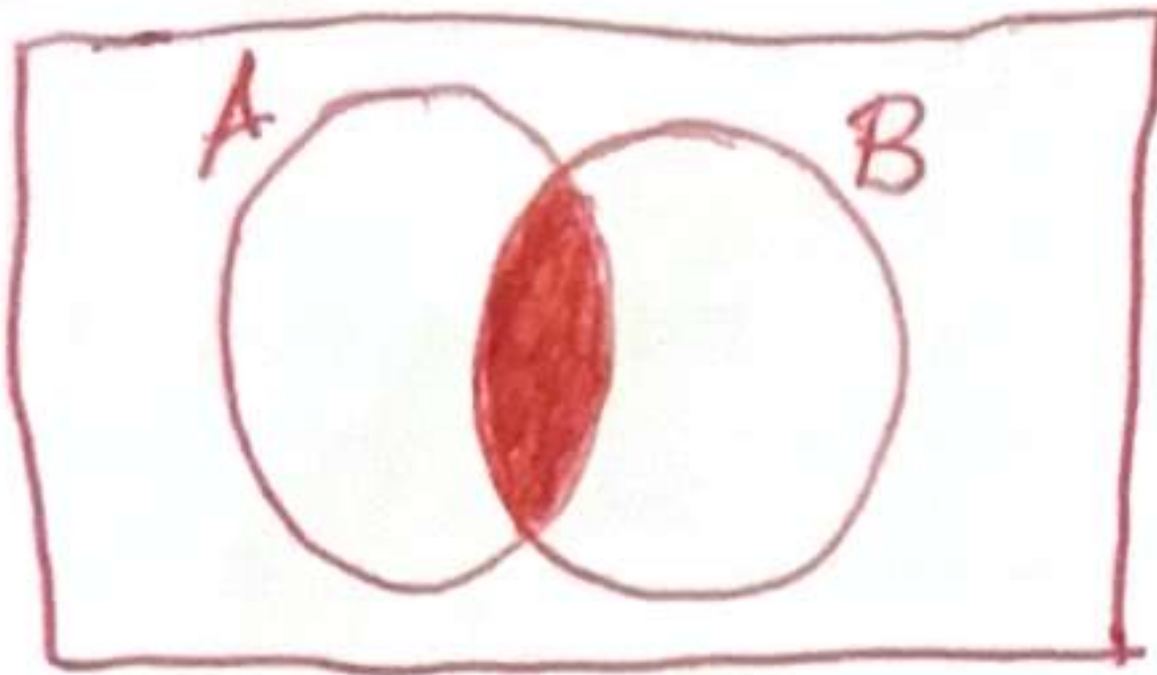
Barcode	fruit_name	price
111	Apple	2
112	Banana	4
113	Orange	3
114	Kiwi	5

inxit

INNER JOIN

- Returns rows where the key exists in both tables.
- We join 2 or more tables on a common column

Syntax: `SELECT` A. column 1 ^{→ I'm taking this column from table 1}
 A. column 2
 B. column 1 ^{→ I'm taking this column from table 2}
`FROM` table1 AS A
`INNER JOIN` table2 AS B
~~`ON A. column`~~
`ON` A. common_column = B. common_column



Customers

customer_id	name
1	Rofhi
2	Mbali

Orders

order_id	customer_id	amount
101	1	300
102	1	150
103	3	500

```
SELECT A.customer_id,  
       A.name,  
       B.amount
```

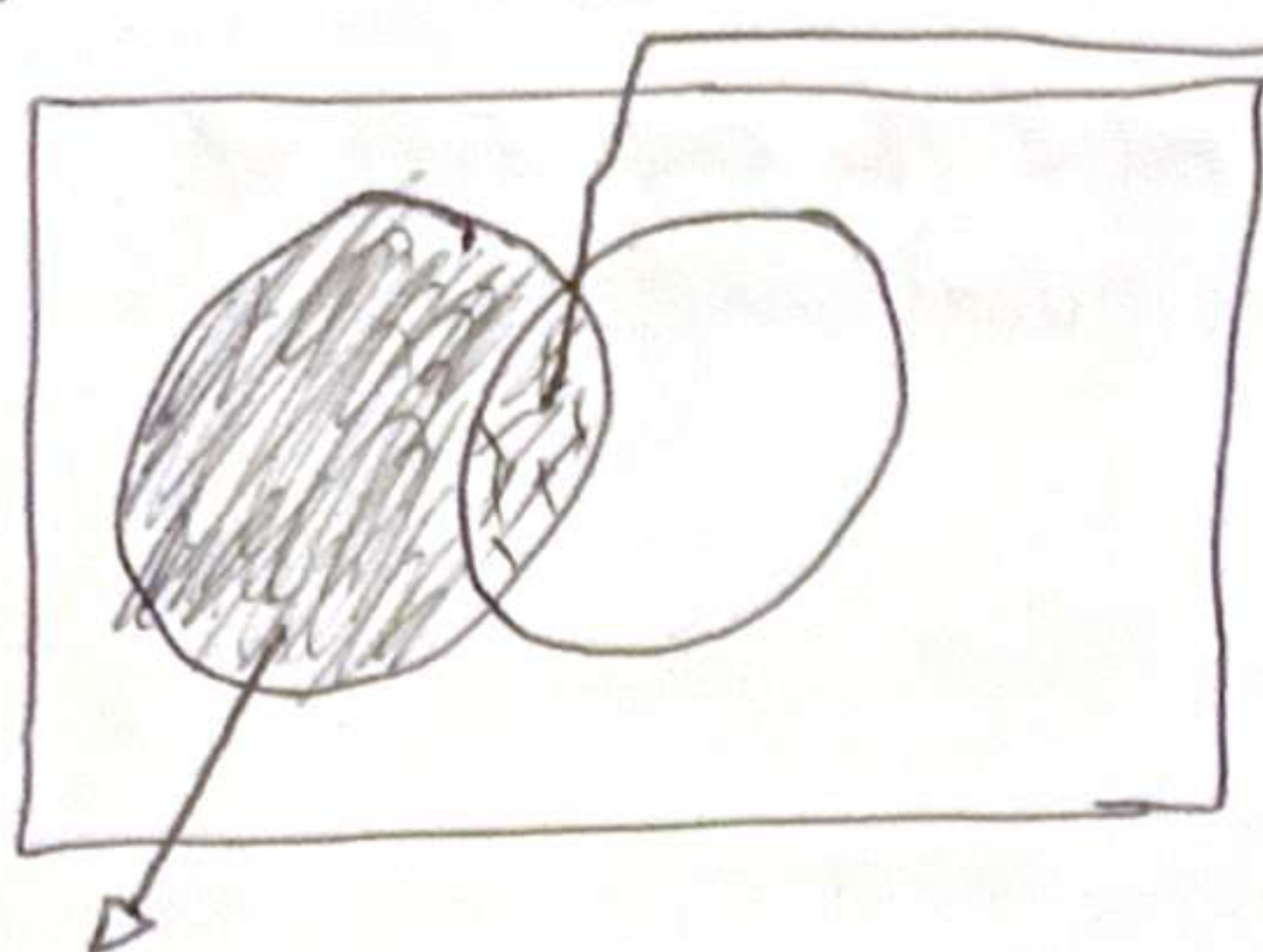
```
FROM customers AS A
```

```
INNER JOIN orders AS B
```

```
ON A.customer_id = B.customer_id;
```

customer_id	name	amount
1	Rofhi	300
2	Rofhi	150

Left LEFT JOIN



I want only the rows that match from the other table.

I want everything every row in the left table i.e base table

SELECT column(s) from the tables you are joining.
 you can choose any column

FROM table 1 AS A

LEFT JOIN table 2 AS B

ON A.common_column = B.column;

Sales customers

customer_id	name
1	Ruthiwa
2	Mvuko

Orders

order_id	customer_id	amount
101	1	300
102	2	150
103	3	500

Write an SQL script to get the customer_id, name, amount

SELECT customer_id,
name,
amount

FROM sales AS

FROM customers AS A

LEFT JOIN orders AS B

ON A.customer_id = B.customer_id;

* Your outcome will contain all the rows in the left table.

customer_id	name	amount
1	Ruthiwa	300
2	Mvuko	150

departments

dept_id	dept_name
10	Sales
20	IT
30	HR
40	Legal

Employees

emp_id	emp_name	dept_id
1	Neo	10
2	Rofhi	20
3	Thabo	20
4	Lesedi	50

I want the emp_dept_id, dept_name, emp_name

SELECT emp_id,
dept_id,
dept_name,
emp_name

If there is no match
we will put a NULL.

FROM ~~employees~~ departments AS A
LEFT JOIN ~~departments~~ employees AS B
ON A.dept_id = B.dept_id;

dept_id	dept_name	emp_name
10	Sales	Neo
20	IT	Rofhi

emp_id	emp_name	dept_id	dept_name
1	Neo	10	Sales
2	Rofhi	20	IT
3	Thabo	20	IT
4	Lesedi	50	Null

Students (base table)

student_id	name
1	Sam
2	Julia
3	Ben

clubs

club_id	student_id	club_name
10	1	Chess
11	2	Debate
12	4	Soccer

I want to know all the students
that play sports, and what sport
they play.

Expected: student_id, name, club_name

SELECT student_id,
name,
club_name

FROM students AS A

LEFT JOIN clubs AS B

ON A.student_id = B.student_id;

student_id	name	club_name
1	Sam	Chess
2	Julia	Debate
3	Ben	Null

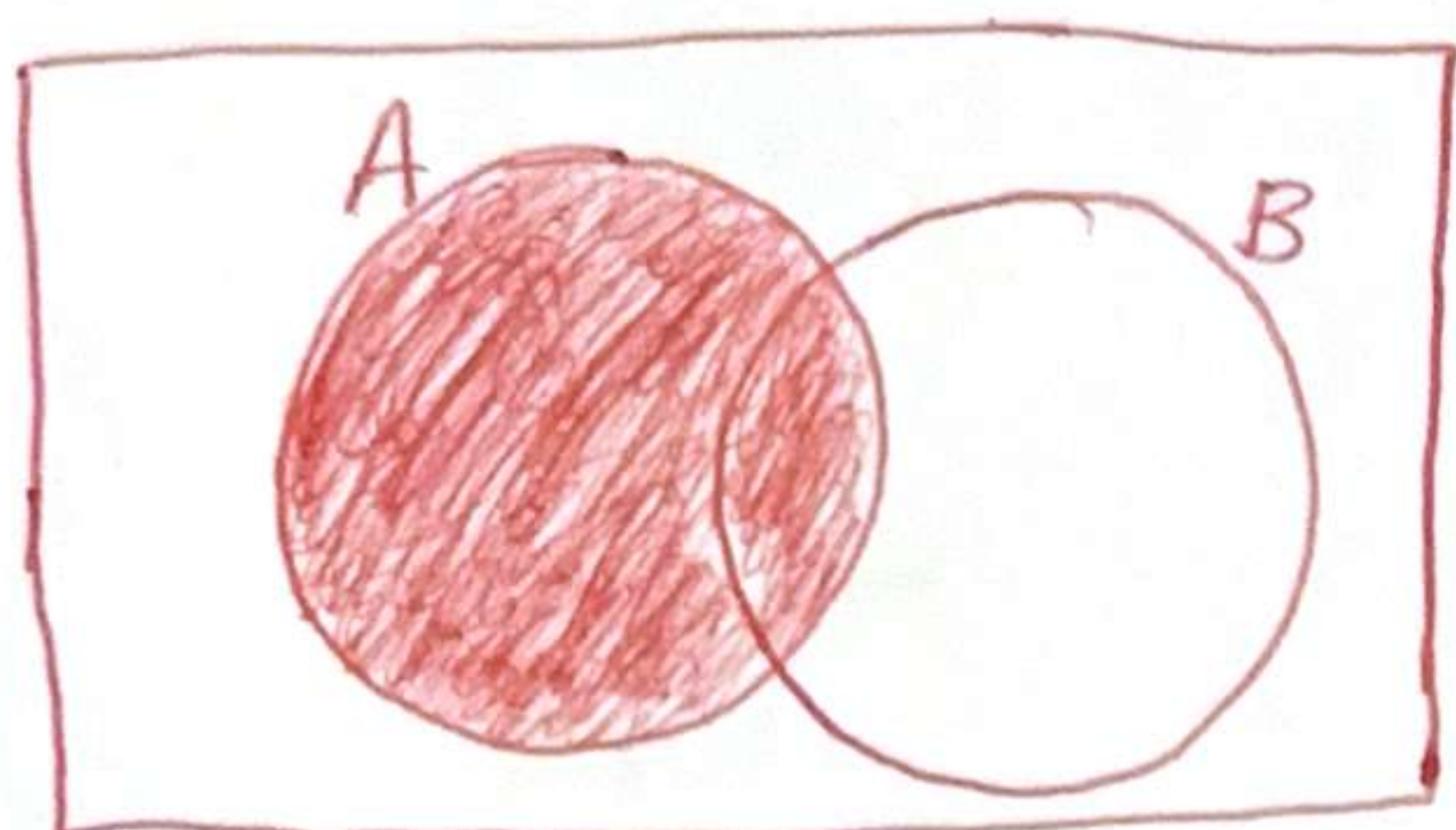
Null → empty space

— Unknown value
— missing value
— blank

LEFT JOIN

→ Returns everything from the left table and adds matches from the right table.

→ If no matches, right sides becomes NULL



Returns everything on the left table

Syntax: `SELECT column`
`FROM table1 AS A` → left table
`LEFT JOIN table2 AS B` → Right table
`ON A.common_column = B.common_column;`

```
SELECT A.customer_id,  
       A.name,  
       B.amount
```

```
FROM customers AS A → left table i.e base table
```

```
LEFT JOIN orders AS B
```

```
ON A.customer_id = B.customer_id;
```

customer_id	name	amount
1	Rofhi	300
1	Rofhi	150
2	Abali	NULL

Questions

1. SQL JOINS

```
SELECT EmployeeID,  
       FirstName,  
       LastName,  
       Department,  
       Salary,  
       ProjectID,  
       ProjectName,  
       Budget,  
       Status
```

From Employees AS A

INNER JOIN ~~Employees~~ ^{Projects} AS B

~~ON A.Common~~

ON A.EmployeeID = B.EmployeeID;

```
SELECT EmployeeID,  
       FirstName,  
       LastName,  
       Department,  
       Salary,  
       ProjectID,  
       ProjectName,  
       Budget,  
       Status
```

FROM Employees AS A

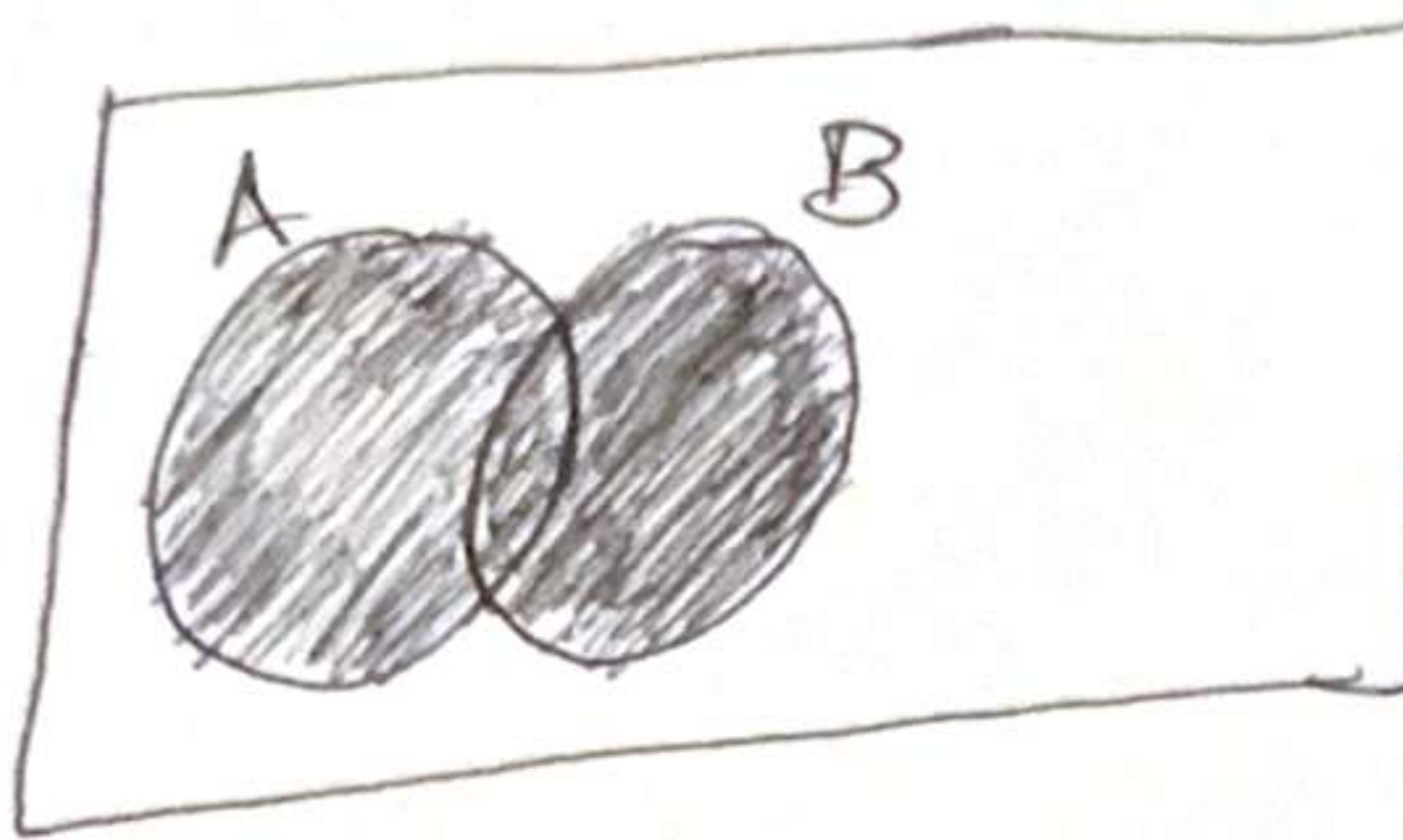
LEFT JOIN Projects AS B

ON A.EmployeeID = B.EmployeeID;

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	PN	B	Status
1	John	Doe	IT	70000	101	AI base	100000	Comp
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing

Full OUTER JOIN

- Returns all the rows from both the tables.



A Full OUTER JOIN returns everything from both tables:

- If a row matches (same key on both sides)
↳ it combines them into one row
- If a row exists only in the table left table - It still show up, with NULL for the right table columns.

- Because it's a join you can select any column from the tables you are joining

Syntax: `SELECT column(s)`

`FROM table1 AS A;`

`Full OUTER JOIN table2 AS B`

`ON A.common_column = B.common_column;`


```
SELECT EmployeeID,  
       FirstName,  
       LastName,  
       Department,  
       Salary,  
       ProjectID,  
       ProjectName,  
       Budget,  
       Status
```

```
FROM Employees AS A  
Full OUTER JOIN Projects AS B  
ON A.EmployeeID = B.EmployeeID;
```

script
er-id,

all
2.